

SORTING THE CARTESIAN PRODUCT

A. V. Aho, T. G. Szymanski, M. Yannakakis

Bell Laboratories

Murray Hill, New Jersey

Summary

1. Introduction

This paper considers a basic vector enumeration problem that has several diverse applications. Let $\{X_j \mid 1 \leq j \leq d\}$ be a collection of sets of items, each item having an associated cost. For convenience, we can assume each set has exactly n items. From this collection we can form n^d d -dimensional vectors by choosing for the j th component of a vector some element from set X_j , for $1 \leq j \leq d$. The cost of a vector is the sum of the costs of its components. Our problem is to enumerate the first k vectors in order of increasing cost.

This problem arises in many different areas of computer science. For example, the best known algorithm for the knapsack problem has at its core a two-dimensional vector enumeration procedure [SS]. The two-dimensional problem also occurs during the multiplication of sorted sparse polynomials when terms must be generated in monotonic order of exponents [AHU].

Applications in which the d -dimensional problem arises occur in the automatic recognition of both printed [C] and spoken [ALS] text in which individual letters or phonemes are often imperfectly recognized and are treated as sets of possible letters with associated likelihoods. In these applications the vectors represent strings of letters and the goal is to find the most likely vector that is actually a word.

2. Background

Previously published algorithms for solving this problem have time and space requirements that are excessive for many practical applications. Harper et al. present an $O(n^2 \log n)$ time algorithm for enumerating the entire set of n^2 vectors for the case $d = 2$ [HPSS].

This algorithm can be easily generalized to produce the first k vectors of the d -dimensional case in $O(kd \log n + n^{d-1})$ time using $O(n^{d-1})$ space. The large $O(n^{d-1})$ initialization time and space requirement make this algorithm particularly undesirable for those applications where only a few vectors need to be enumerated.

Johnson and Mizoguchi give an algorithm to select the k th element in $X_1 + X_2 + \dots + X_d$ that runs in $O(dn^{d/2} \log n)$ time and $O(n^{d/2})$ space [JM]. For the two-dimensional case Frederickson and Johnson give a more efficient $O(n + c \log(k/c))$ time algorithm for selecting the k th element in $X_1 + X_2$. Here c is the smaller of n and k .

Fredman gives an information-theoretic argument which can be extended to show that for any fixed n and d , a height $O(n^d)$ decision tree exists for producing the entire enumeration of n^d vectors [F]. Unfortunately, the argument is non-constructive and it is still not known whether a uniform $O(n^d)$ time algorithm exists for this problem. Interesting discussions of various related aspects of this problem are contained in [JK, GMPR].

3. Algorithms for Enumerating the First k Vectors

We begin by observing that the problem of determining whether a vector of a given cost exists somewhere in the enumeration is NP-complete. So too is the problem of producing the immediate successor of a given vector in the enumeration. Johnson and Mizoguchi have also noted that the existence of a polynomial (in n and d) time algorithm for selecting the k th vector would imply $P = NP$ [JM].

These results suggest that efficient algorithms to generate arbitrary subsequences of the enumeration are not likely to exist. They also suggest that no constant space algorithm can produce each succes-

sive vector in a constant amount of time, that is, it is not possible to have an $O(k)$ time and $O(d)$ space algorithm.

We can, however, give the following algorithms to generate the first k vectors of the enumeration:

- (1) an $O(dk \log k)$ time and $O(dk)$ space algorithm,
- (2) an $O(k(d + \log n))$ time and $O(d(n+k))$ space algorithm, and
- (3) an $O(k^2)$ time and $O(d)$ space algorithm.

The first algorithm is easy to implement using a priority queue. The second is more complicated but, to the best of our knowledge, is the fastest known general algorithm for enumerating the cartesian product. The third algorithm has the advantage of using constant space to generate the first k elements.

In addition we show how dynamic programming can be used to generate in linear time an enumeration that is almost monotonic. If δ is the largest difference between the costs of consecutive valued items in any X_j , we can produce an enumeration $v_1, v_2, \dots, v_n$ which has the property that $c(v_i) - \frac{\delta}{n^{d-1}} \leq c(v_j)$ whenever $i \leq j$. The algorithm is particularly simple and is useful in those applications such as text recognition where a strictly monotonic enumeration is not needed.

4. References

- [AHU] Aho, A. V., J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Addison-Wesley, Reading, Mass., 1974.
- [ALS] Aldefeld, B., S. E. Levinson, and T. G. Szymanski, "A Minimum-Distance Search Technique and its Application to Automatic Directory Assistance," *BSTJ*, to appear.
- [C] Cherry, L. L., "Experiments in Multifont Character Recognition," Bell Laboratories, Murray Hill, New Jersey.
- [F] Fredman, M. L., "How Good is the Information Theory Bound in Sorting?" *Theoretical Computer Science* 1 (1976), 355-361.
- [FJ] Frederickson, G. N., and D. B. Johnson, "Optimal Algorithms for Generating Quantile Information in $X+Y$ and Matrices with Sorted Columns," *Proc. 1979 Conference on Information Sciences and Systems*, Johns Hopkins University, pp. 47-52.
- [GMPR] Guibas, L. J., E. M. McCreight, M. F. Plass, and J. R. Roberts, "A New Representation for Linear Lists," *Proc. Ninth Annual ACM Symposium on Theory of Computing*, May 1977, pp. 49-60.
- [HPSS] Harper, L. H., T. H. Payne, J. E. Savage, and E. Straus, "Sorting $X+Y$," *Comm. ACM* 18:6, (June, 1975), 347-349.
- [JK] Johnson, D. B., and S. D. Kashdan, "Lower Bounds for Selection in $X+Y$ and Other Multisets," *J. ACM* 25:4 (October, 1978), 556-570.
- [JM] Johnson, D. B., and T. Mizoguchi, "Selecting the K th Element in $X+Y$ and $X_1+X_2+\dots+X_m$," *SIAM J. Computing* 7:2 (May, 1978), 147-153.
- [SS] Schroepfel, R., and A. Shamir, "A $T \cdot S^2 = O(2^n)$ Time/Space Tradeoff for Certain NP-Complete Problems," *Proc. 20th Annual IEEE Symposium on Foundations of Computer Science*, October, 1979, pp. 328-336.